PYTHON FUNDAMENTALS

Python Loop Practice Workbook

50 progressively structured questions using your learned Python topics plus for, while, range(), break, and continue.

15	15	15	5
EASY	MEDIUM	HARD	COMPLEX

What you'll practise

Conditions	if, else, nested if
Strings	endswith(), capitalize(), replace(), find(), count(), isalpha(), isdigit()
Lists	append(), sort(), reverse(), insert(), remove(), pop()
Sets	add(), remove(), clear(), pop(), union(), intersection()
Loop toolkit	for, while, range(), break, continue

How to use this workbook

1. Read the task and requirements before writing code.
2. Solve in VS Code without searching for a complete solution.
3. Test the sample case and then try another value or edge case.
4. Mark the tracker only when you can explain your logic.

Questions only - no solution code is included intentionally.

Easy

Build confidence with loop structure, counters, and one straightforward operation.

DIRECT LOOP PRACTICE

E01 Print a number sequence — FOR

Use a for loop with range() to print the integers from 1 through 10, one per line.

Requirements: Use for and range(); print one integer per line.

Test: Expected output: 1 through 10, one per line.

My logic: _____

E02 Print from 1 through n — FOR

Assume n = 12. Use for and range() to print the integers from 1 through n.

Requirements: The ending value n must be included.

Test: n = 12 | Expected: 1, 2, 3, ... 12.

My logic: _____

E03 Count upward with while — WHILE

Use a while loop to print the integers from 1 through 10. Start with counter = 1 and update it correctly.

Requirements: Start at 1; update the counter on every round.

Test: Expected output: 1 through 10, one per line.

My logic: _____

E04 Count downward — WHILE

Assume n = 8. Use while to print n, then n - 1, and continue down to 1.

Requirements: Stop after printing 1; do not print 0.

Test: n = 8 | Expected: 8, 7, 6, 5, 4, 3, 2, 1.

My logic: _____

E05 Running total — FOR

Use for and range() to calculate and print the total of the integers from 1 through 10. Use a variable to keep the running total.

Requirements: Update a total variable inside the loop.

Test: Expected total: 55.

My logic: _____

Easy / Continued

Continue with the same loop and syllabus rules.

DIRECT LOOP PRACTICE

E06 Print even numbers — FOR

Use range() to print all even integers from 2 through 20.

Requirements: Print only even values; include 2 and 20.

Test: Expected: 2, 4, 6, ... 20.

My logic: _____

E07 Visit every list item — FOR

Given numbers = [4, 8, 2, 9, 5], use a for loop to print each list element.

Requirements: Visit each element once and keep the original order.

Test: Expected: 4, 8, 2, 9, 5.

My logic: _____

E08 Countdown by fives — WHILE

Assume n = 25. Use while to print the current value and subtract 5 each round. Stop when n reaches 0.

Requirements: Print the current value before subtracting 5.

Test: Expected: 25, 20, 15, 10, 5, 0.

My logic: _____

E09 Print each character — FOR

Given word = 'Python', use a for loop to print one character per line.

Requirements: Loop directly through the string.

Test: Expected characters: P, y, t, h, o, n.

My logic: _____

E10 Count alphabetic characters — FOR

Given text = 'BCA2026!', use a for loop and isalpha() to count how many characters are alphabetic.

Requirements: Increase a counter only when isalpha() is true.

Test: text = BCA2026! | Expected count: 3.

My logic: _____

Easy / Continued

Continue with the same loop and syllabus rules.

DIRECT LOOP PRACTICE

E11 Capitalize every word — FOR

Given words = ['hello', 'WORLD', 'python'], use a for loop to print each word after applying capitalize().

Requirements: Apply capitalize() to each word as you loop.

Test: Expected: Hello, World, Python.

My logic: _____

E12 Find Python filenames — FOR

Given filenames = ['notes.txt', 'app.py', 'photo.png', 'main.py'], use a for loop and endswith('.py') to print only the Python filenames.

Requirements: Check each filename with endswith('.py').

Test: Expected: app.py, main.py.

My logic: _____

E13 Count digit characters — FOR

Given text = 'a1b2c3d4', use a for loop and isdigit() to count the digit characters.

Requirements: Count characters for which isdigit() is true.

Test: text = a1b2c3d4 | Expected count: 4.

My logic: _____

E14 Subtract three each round — WHILE

Assume n = 30. Use while to print n, then subtract 3 each round. Continue while the current value is positive.

Requirements: Check that the current value is positive before each round.

Test: Expected: 30, 27, 24, ... 3.

My logic: _____

E15 Visit every set element — FOR

Given values = {2, 4, 6, 8}, use a for loop to print every set element. The output order does not matter.

Requirements: Loop through the set; do not expect a fixed order.

Test: Expected elements: 2, 4, 6, 8 in any order.

My logic: _____

Medium

Combine loops with one or two learned conditions, string methods, or collection me

COMBINE 1-2 LEARNED IDEAS

M01 Collect even numbers — FOR

Given numbers = [12, 7, 18, 5, 20, 9], use for, if, and append() to create and print a new list containing only the even values.

Requirements: Use if to check parity and append() to build the result list.

Test: Expected result: [12, 18, 20].

My logic:

M02 Print multiples of three — WHILE

Start with counter = 1. Use while to print only the multiples of 3 from 1 through 20. Increase the counter on every round.

Requirements: Check divisibility by 3; always increase the counter.

Test: Expected: 3, 6, 9, 12, 15, 18.

My logic:

M03 Count letters and digits — FOR

Given text = 'BCA2026!', use a for loop, isalpha(), and isdigit() to count alphabetic characters and digits separately.

Requirements: Maintain separate counters; ignore other characters.

Test: text = BCA2026! | Expected: 3 letters, 4 digits.

My logic:

M04 Collect words ending in ing — FOR

Given words = ['apple', 'running', 'coding', 'book'], use for, endswith('ing'), and append() to build a new list containing only words that end in ing.

Requirements: Test endswith('ing') before appending.

Test: Expected result: [running, coding].

My logic:

M05 Insert capitalized words — FOR

Given words = ['alice', 'BOB', 'python'], use for, capitalize(), and insert() to build a new list. Insert each capitalized word at position 0, then print the list.

Requirements: Apply capitalize() and insert each item at index 0.

Test: Expected result: [Python, Bob, Alice].

My logic:

Medium / Continued

Continue with the same loop and syllabus rules.

COMBINE 1-2 LEARNED IDEAS

M06 Build a descending list

WHILE

Start with n = 50 and values = []. While n is positive, append its current value to values and subtract 10. Print the final list.

Requirements: Append before subtracting 10; stop when n is no longer positive.

Test: Expected result: [50, 40, 30, 20, 10].

My logic:

M07 Skip negative numbers

FOR

Given numbers = [-3, 4, -1, 8, 0, 5], use for and continue to skip negative numbers and print every other value.

Requirements: Use continue for negative values; keep zero and positive values.

Test: Expected: 4, 8, 0, 5.

My logic:

M08 Stop at the first multiple of seven

FOR

Use range() to check integers from 1 through 50. Stop at the first number divisible by 7 using break, and print the number that caused the loop to stop.

Requirements: Use break at the first match.

Test: Expected stopping value: 7.

My logic:

M09 Find words containing app

FOR

Given words = ['banana', 'band', 'apple', 'application'] and target = 'app', use find() in a for loop to print only the words in which the target is found.

Requirements: Treat a find() result of -1 as not found.

Test: Expected: apple, application.

My logic:

M10 Inspect five entered words

WHILE

Use a while loop to read 5 words with input(). For each word, print how many times it contains 'a' using count(), and print the first position of 'n' using find().

Requirements: Process exactly 5 inputs; use count() and find() for each.

Test: Input word banana | Expected: count of a = 3; first n position = 2.

My logic:

Medium / Continued

Continue with the same loop and syllabus rules.

COMBINE 1-2 LEARNED IDEAS

M11 Remove and sort values

FOR

Given numbers = [9, 3, 7, 1, 5], first remove 7 using remove(). Then use sort() and a for loop to print the remaining values in ascending order.

Requirements: Call remove(7), then sort(), then loop through the list.

Test: Expected: 1, 3, 5, 9.

My logic:

M12 Calculate a descending sum

WHILE

Start with n = 10 and total = 0. While n is positive, add n to total and decrease n by 1. Print the final total.

Requirements: Add the current n before decreasing it.

Test: Expected total: 55.

My logic:

M13 Replace text inside words

FOR

Given words = ['catnap', 'catalog', 'dog', 'scatter'], use replace('cat', 'dog') and append() in a for loop to store the changed words in a new list.

Requirements: Replace the substring in each word, then append the changed word.

Test: Expected result: [dognap, dogalog, dog, sdogter].

My logic:

M14 Display a set union

FOR

Given set_a = {1, 2, 3} and set_b = {3, 4, 5}, use union() to create a combined set, then use for to display its elements.

Requirements: Create the union first; output order does not matter.

Test: Expected elements: 1, 2, 3, 4, 5.

My logic:

M15 Display common set values

FOR

Using the same set_a and set_b, use intersection() to create a set of common elements, then use for to display them.

Requirements: Create the intersection and loop through its result.

Test: Expected common element: 3.

My logic:

Hard

Coordinate multiple operations and make the order of loop updates matter.

COMBINE 3-4 LEARNED IDEAS

H01 Clean and sort valid names

FOR

Given names = ['alice', 'BOB', 'charlie7', 'diana'], use for and isalpha() to keep only alphabetic names. Apply capitalize(), add each valid name to a new list with append(), then sort() the list.

Requirements: Reject names containing non-letters; capitalize, append, and sort valid names.

Test: Expected result: [Alice, Bob, Diana].

My logic:

H02 Collect six entries into a set

WHILE

Use while to read exactly 6 entries with input(). Skip non-alphabetic entries with continue. For each valid entry, apply capitalize() and add it to a set with add(). Count all 6 entries, including skipped ones.

Requirements: The counter must include invalid entries; continue must not skip the counter update.

Test: Inputs cat, BOB, 123, hi, hi, good | Set: {Cat, Bob, Hi, Good}; count: 6.

My logic:

H03 Filter until a zero appears

FOR

Given numbers = [12, -4, 0, 7, 18, -2, 25], use for to skip negative values with continue, stop when you reach 0 with break, and append() earlier positive values to a new list. Then sort() and print that list.

Requirements: Skip negatives; stop at zero; only earlier positive values enter the result.

Test: Expected result: [12].

My logic:

H04 Clean words and keep ing endings

FOR

Given words = ['tablet!', 'coding!', 'testing!', 'python!'], use replace('!', '') to clean each word. If the cleaned word endswith('ing'), add it to a result set using add().

Requirements: Replace punctuation before checking endswith(); use a set for results.

Test: Expected set: {coding, testing}.

My logic:

H05 Skip and stop during accumulation

WHILE

Start with n = 1 and total = 0. While n is at most 30, stop with break if n is divisible by 7. Skip multiples of 4 with continue. Add all other values to total. Make sure n increases on every non-break round, including rounds that use continue.

Requirements: Check break first; ensure n is updated even when continue is used.

Test: Loop stops at 7; expected total before stopping: 17.

My logic:

Hard / Continued

Continue with the same loop and syllabus rules.

COMBINE 3-4 LEARNED IDEAS

H06 Separate letters and digits — FOR

Given text = 'PyTh0n2026!', use for to build two new strings: one containing only alphabetic characters and one containing only digits. Use isalpha() and isdigit() to decide where each character goes. Ignore all other characters.

Requirements: Use two result strings; ignore punctuation and symbols.

Test: Expected strings: PyThn and 02026.

My logic:

H07 Find, sort, and reverse matches — FOR

Given words = ['banana', 'band', 'apple', 'application', 'bat'] and target = 'app', use find() to identify matching words. Add matches to a new list with append(), then apply sort() followed by reverse().

Requirements: Append only words containing target; sort before reverse.

Test: Expected result: [application, apple].

My logic:

H08 Create a resettable set collector — WHILE

Use while True to keep reading entries with input(). If the entry is 'STOP', end the loop with break. If it is 'CLEAR', empty the set using clear() and continue. Otherwise, add its capitalized form only when isalpha() is true.

Requirements: STOP ends input; CLEAR empties the set; only alphabetic entries are added.

Test: Inputs alpha, CLEAR, bob, STOP | Expected set: {Bob}.

My logic:

H09 Count repeated letters in words — FOR

Given words = ['banana', 'cab', 'apple', 'data', 'sky'], use for and count('a') to add words containing at least two 'a' characters to a new list. Then sort() and print the list.

Requirements: Keep words with count('a') >= 2, then sort.

Test: Expected result: [banana, data].

My logic:

H10 Replace and collect changed words — FOR

Given words = ['cat', 'catch', 'scatter', 'cater', 'dog'], use replace('cat', 'fox') on each word. Use count('fox') to add only changed words to a new list with append(), then sort() the result.

Requirements: Detect replacements with count('fox'); do not include unchanged words.

Test: Expected result: [fox, foxch, foxer, sfoxter].

My logic:

Hard / Continued

Continue with the same loop and syllabus rules.

COMBINE 3-4 LEARNED IDEAS

H11 Empty a set with pop

WHILE

Start with remaining = {2, 4, 6, 8, 10} and removed_count = 0. First remove 6 using the set's remove() method. Then use while to keep removing an arbitrary element with pop() until the set is empty. Print each removed value and the final count. Do not expect a particular removal order.

Requirements: Remove 6 first; use pop() until empty; count each popped item.

Test: Expected final removed_count: 4; removal order can vary.

My logic:

H12 Compare a union with an intersection

FOR

Given set_a = {1, 2, 3, 4} and set_b = {3, 4, 5, 6}, create their union() and intersection(). Loop through the union and print whether each value is common to both sets or belongs to only one set.

Requirements: Use the intersection to decide whether each union value is common.

Test: Common: 3, 4 | Only one set: 1, 2, 5, 6.

My logic:

H13 Build a set with skip and stop rules

WHILE

Start with n = 1 and kept = set(). Use while to examine values up to 30. Stop when n reaches 23 using break; skip multiples of 5 using continue; add every other value to kept. Update n correctly on every non-break round.

Requirements: Stop before adding 23; skip multiples of 5; update n on continue rounds.

Test: The set contains 1-22 except 5, 10, 15, and 20.

My logic:

H14 Filter matching PDF filenames

FOR

Given files = ['report_v1.pdf', 'report_final.pdf', 'notes.pdf', 'report.txt', 'old_report.pdf'] and target = 'report', use endswith('.pdf') and find(target) to append only matching PDF filenames to a new list. Then sort() and reverse() that list.

Requirements: A filename must satisfy both checks; sort before reverse.

Test: Expected result: [report_v1.pdf, report_final.pdf, old_report.pdf].

My logic:

H15 Pop and filter list values

WHILE

Given numbers = [5, 3, 8, 1, 4] and even_values = [], use while to repeatedly take the last value with pop() until the list is empty. Append even values to even_values, then sort() and print the result.

Requirements: Pop from the original list until empty; keep only even values.

Test: Expected result: [4, 8].

My logic:

Complex Practice

Multi-step challenges within your current syllabus.

MULTI-STEP CHALLENGES / SAME SYLLABUS

C01 Unique-word collector

MIXED

Build a small unique-word collector. Use while to read entries with input() until the user enters 'STOP'; use break to end. If the user enters 'CLEAR', empty the set with clear() and skip to the next entry using continue. For any other entry, use isalpha() to reject invalid entries; add valid entries in capitalize() form with add(). After the loop, use for to display the set.

Requirements: Use while for input and for for display; support STOP, CLEAR, and alphabetic-only entries.

Test: Inputs alice, 123, CLEAR, Maya, STOP | Expected set: {Maya}.

My logic:

C02 Filtered even-number collector

FOR

Create a number filter using range(1, 51). Stop when the current number is greater than 35 using break. Skip multiples of 3 using continue. Add the remaining even numbers to a set with add(), and keep a separate counter of how many numbers were added. At the end, display the set and the counter.

Requirements: Use break, continue, a set, and a separate counter.

Test: Expected count: 12; set: {2, 4, 8, 10, 14, 16, 20, 22, 26, 28, 32, 34}.

My logic:

C03 Classify codes into three lists

FOR

Given codes = ['AB12', 'hello', '2026', 'A1B2', 'data7', 'PYTHON'], classify every code into one of three lists: alphabetic, numeric, or mixed. Use isalpha() and isdigit() to decide, and append() to store each code. Finally, sort() each list and display all three.

Requirements: A code is alphabetic, numeric, or mixed; sort each result list.

Test: Alphabetic: [PYTHON, hello] | Numeric: [2026] | Mixed: [A1B2, AB12, data7].

My logic:

Complex Practice / Continued

Continue with the same loop and syllabus rules.

MULTI-STEP CHALLENGES / SAME SYLLABUS

C04 Split popped values into even and odd

WHILE

Given numbers = [5, 3, 8, 1, 4, 7, 2], start with even_values = set() and odd_values = []. While the original list is not empty, take one value using pop(). Add even values to the set with add() and append odd values to the list. At the end, sort and display the odd list, then use for to display the even set. Do not assume a fixed set order.

Requirements: Empty the original list using pop(); route values by parity; sort only the odd list.

Test: Odd list: [1, 3, 5, 7] | Even set: {2, 4, 8}.

My logic:

C05 Clean and filter matching words

FOR

Given words = ['banana!', 'band', 'apple', 'application', 'bat', 'banana!'] and target = 'app', process each word in a for loop. First remove '!' using replace(). Keep a cleaned word only when find(target) shows that the target is present and count('a') is at least 2. Add matching words to a new list with append(), then sort() and reverse() the final list.

Requirements: Clean first; require both find(target) and count('a') >= 2; then sort and reverse.

Test: Expected result: [application].

My logic:

Practice Tracker

Mark a question only when you can solve it and explain why your logic works.

QUESTION	STATUS	DATE
E01		
E02		
E03		
E04		
E05		
E06		
E07		
E08		
E09		
E10		
E11		
E12		
E13		
E14		
E15		
M01		
M02		
M03		
M04		
M05		
M06		
M07		
M08		
M09		
M10		

QUESTION	STATUS	DATE
M11		
M12		
M13		
M14		
M15		
H01		
H02		
H03		
H04		
H05		
H06		
H07		
H08		
H09		
H10		
H11		
H12		
H13		
H14		
H15		
C01		
C02		
C03		
C04		
C05		

Quick reference

for loop

```
for value in values:
    print(value)
```

range(start, stop) excludes stop.

continue skips to the next round.

while loop

```
while condition:
    # update condition
```

break exits the loop immediately.

Update while counters to avoid an infinite loop.

Tracker key: R = revised | H = solved with a hint | I = solved independently.